

1) Write a C++ program that dynamically allocates an array based on user input, populates it, and sorts it in ascending order. You must adhere to strict constraints regarding memory management and array manipulation.

Requirements & Constraints:

1. **Dynamic Allocation & Pass-by-Reference:** Create a function named `createAndPopulateArray`. This function must take an integer pointer by reference (e.g., `int*&`) and an integer size. It should ask the user for the array size, dynamically allocate memory for the array on the heap, and populate it with user input.
2. **Strict Pointer Arithmetic:** Standard array indexing (e.g., `arr[i]`) is strictly forbidden throughout the entire program. All input reading, array traversal, value comparisons, and value swapping must be performed using pointer arithmetic (e.g., `*(arr + i)`).
3. **Sorting Function:** Create a function named `sortArray` that takes the integer pointer and the size of the array. Implement a sorting algorithm (like Bubble Sort) using only pointers to compare and swap the elements.
4. **Memory Management:** The program must not have any memory leaks. Ensure the dynamically allocated memory is properly freed in the `main()` function after the sorted array is displayed.

2) Write a menu-driven C++ program that performs recursive operations on numbers:

1. Find the factorial of a number
2. Find the GCD of two numbers
3. Find the LCM of two numbers (use GCD recursively)
4. Find the sum of digits of a number
5. Reverse a number
6. Exit

The program should display a menu to the user, take input, and perform the selected operation.

3. Implement a Stack Using Queues

Task: Design and implement a LIFO (Last-In-First-Out) Stack data structure using only standard FIFO (First-In-First-Out) Queues from scratch.

- Constraints: You may use either one or two queues.
- Operations to implement:
 - `void push(int x)`: Pushes element `x` to the top of the stack.

- `int pop()`: Removes the element on the top of the stack and returns it.
- `int top()`: Returns the element on the top of the stack without removing it.
- `bool empty()`: Returns true if the stack is empty, false otherwise.
- *Follow-up challenge*: Can you make the push operation strictly $O(1)$ time complexity, and handle the heavy lifting inside the pop operation?

4) Write a C++ program to perform operations on a singly linked list using recursion.

Requirements:

1. Define a Node structure: `struct Node { int data; Node* next; };`
2. Implement the following recursive functions:
 - `Node* insert(Node* head, int value)`
 - Recursively insert a new node at the end of the linked list.
 - `Node* sortLinkedList(Node* head)`
 - Recursively sort the linked list in ascending order.
 - `void display(Node* head)`
 - Recursively print all elements of the linked list.
3. In the `main()` function:
 - Ask the user to input the number of elements.
 - Insert elements into the linked list.
 - Display the original list.
 - Sort the linked list recursively.
 - Display the sorted list

5. The Corporate Hierarchy

Task: Design a basic payroll system using inheritance.

- Base Class (Employee):
 - Create an abstract class with protected attributes for name and baseSalary.
 - Write a constructor to initialize these two values.
 - Declare a pure virtual function `double calculateSalary() = 0;`
 - *Include a virtual destructor.*
- Derived Classes:
 - Create a Manager class that inherits from Employee. Add a bonus attribute. Override `calculateSalary()` to return `baseSalary + bonus`.

- Create an Intern class that inherits from Employee. Add hourlyRate and hoursWorked attributes. Override calculateSalary() to return hourlyRate * hoursWorked (Interns ignore the base salary).
- The Test (main): Create an array of Employee* pointers. Use new to add one Manager and one Intern to the array. Loop through the array and print their salaries (demonstrating polymorphism). Finally, delete the objects.

6. The Currency Engine (Compile-time Polymorphism & Object Interaction)

Concepts tested: Static Members, Dynamic Memory, Copy Constructor, Method Overloading, Object Interaction, and Pass-by-Reference.

Task: Create a Wallet class to manage money.

- Static Member & Getter: Include a private static int totalWallets that increases by 1 in the constructor and decreases by 1 in the destructor. Create a public static int getTotalWallets() method to return this count.
- Dynamic Memory: Instead of a normal variable, use a dynamically allocated pointer double* balance to store the money.
- Deep Copy & Cleanup: Because you used dynamic memory (new double), you must write a custom Destructor (to delete the memory) and a Copy Constructor (to prevent shallow copy crashes if one wallet is copied to another).
- Method Overloading: Write two addFunds methods. One accepts a double (e.g., 5.50) and one accepts an int (e.g., 5). Both should add the amount to the balance.
- Spending Funds: Write a bool spendFunds(double amount) method. It should check if there is enough money. If yes, deduct the amount and return true. If not, print "Insufficient funds" and return false.
- Object Interaction: Write a bool transferTo(Wallet& receiver, double amount) method. This method should attempt to deduct the amount from the current wallet's balance and add it to the receiver wallet's balance, returning true if successful.
- Display Method: Write a displayBalance() method to print the wallet's current balance to the console.